

Cross Validation Study for a Single Stint

Cole Cappello

9-7-25

```
set.seed(434)

library(mosaic)
library(tidyverse)
library(effects)
library(ggResidpanel)
library(catstats2)
library(Matrix)
library(KFAS)
library(rstan)

options(mc.cores = parallel::detectCores())
```

Simulation and Code

Here we'll simulate from the model

$$y_t = \alpha_t + \epsilon_t \quad (1)$$

$$\alpha_{t+1} = \alpha_t + \nu_t + \omega_t \quad (2)$$

$$\nu_{t+1} = \nu_t + \beta \quad (3)$$

where $\epsilon_t \sim N(0, .3)$, $\beta = .01$, $\nu_1 = .01$, $\omega_t \sim N(0, .1)$, and $\alpha_1 = 96$.

```
# Number of states
N <- 25

# Beta
beta <- .01

# Initialize nu vector
nu <- rep(NA, N)
nu[1] <- .01

# Initialize alpha vector
alpha <- rep(NA, N)
alpha[1] <- 96

# Initialize process errors
omega <- rnorm(n = N, mean = 0, sd = .1)
```

```

# Store true latent states
for(i in 2:(length(alpha))) {
  alpha[i] <- alpha[i-1] + nu[i-1] + omega[i]
  nu[i] <- nu[i-1] + beta
}

# Vector of observation errors
epsilon <- rnorm(n = N, mean = 0, sd = .3)

# Simulated observations
y <- alpha + epsilon

```

Forward Chaining Cross-Validation

The problem with Cross-Validation in this setting is that we have a pretty small sample size. Typically, a stint will be between 10 and 25 laps. For the 25 lap case above, the way I'd like to implement CV will be like so:

```

fold 1: training [laps 1-20], test [lap 21]
fold 2: training [laps 1-21], test [lap 22]
fold 3: training [laps 1-22], test [lap 23]
fold 4: training [laps 1-23], test [lap 24]
fold 5: training [laps 1-24], test [lap 25]

```

This seems like the best method out of those that I have come across for doing cross validation on a small sample size time series.

I could also potentially decrease the initial training size (maybe 15 instead of 20).

To estimate the one step ahead prediction, I have included code in the generated quantities block that will draw a predicted state and predicted observation for each posterior draw.

Lastly, the summary statistic we'll use is the Squared Prediction Error:

$$(y_{T+1} - \hat{y}_{T+1})^2$$

where $\hat{y}_{T+1}$ is the mean of the `y_pred` generated quantities.

CV Function Definition

```

## This chunk will define a function that takes in a stan model and then performs the above cross validation

stint_CV <- function(model_string, data_list) {

  # Number of folds
  K <- 5

  stan.models <- vector("list", length = K)
  posteriors <- vector("list", length = K)

  # Vector to Store one-step ahead predictions
  ypred_hat <- rep(NA, K)

```

```

for(i in 1:K) {

  # Update data vector to allow for next step prediction
  data_list[["TT"]] <- length(y[1:(i+N-K-1)])
  data_list[["y"]] <- y[1:(i+N-K-1)]

  # Fit model on subset of data then get one step ahead prediction
  stan.models[[i]] <- stan(file = model_string,
                           data = data_list,
                           chains = 3, iter = 30000,
                           control = list(adapt_delta = .99))
  posteriors[[i]] <- extract(stan.models[[i]])
  ypred_hat[i] <- mean(posteriors[[i]]$y_pred)

}

# Calculate Summary Statistic
MSPE <- mean((y[(N-K+1):N] - ypred_hat)^2)
RMSPE <- sqrt(MSPE)

plot_cv <- plot(x = ypred_hat, y = y[(N-K+1):N],
                xlab = "Predictions",
                ylab = "True Observations")

return(list(RMSPE, plot_cv))

}

# list(TT = length(y[1:(i+N-K-1)]), y = y[1:(i+N-K-1)], z0 = 96, v0 = .01, sdo0 = .3)

```

Cross Validation for Various Models

Here are the 3 models that we'll do cross validation for:

$$y_t = \alpha_t + \epsilon_t \quad (4)$$

$$\alpha_{t+1} = \alpha_t + \nu \quad (5)$$

$$y_t = \alpha_t + \epsilon_t \quad (6)$$

$$\alpha_{t+1} = \alpha_t + \nu + \omega_t \quad (7)$$

$$y_t = \alpha_t + \epsilon_t \quad (8)$$

$$\alpha_{t+1} = \alpha_t + \nu_t + \omega_t \quad (9)$$

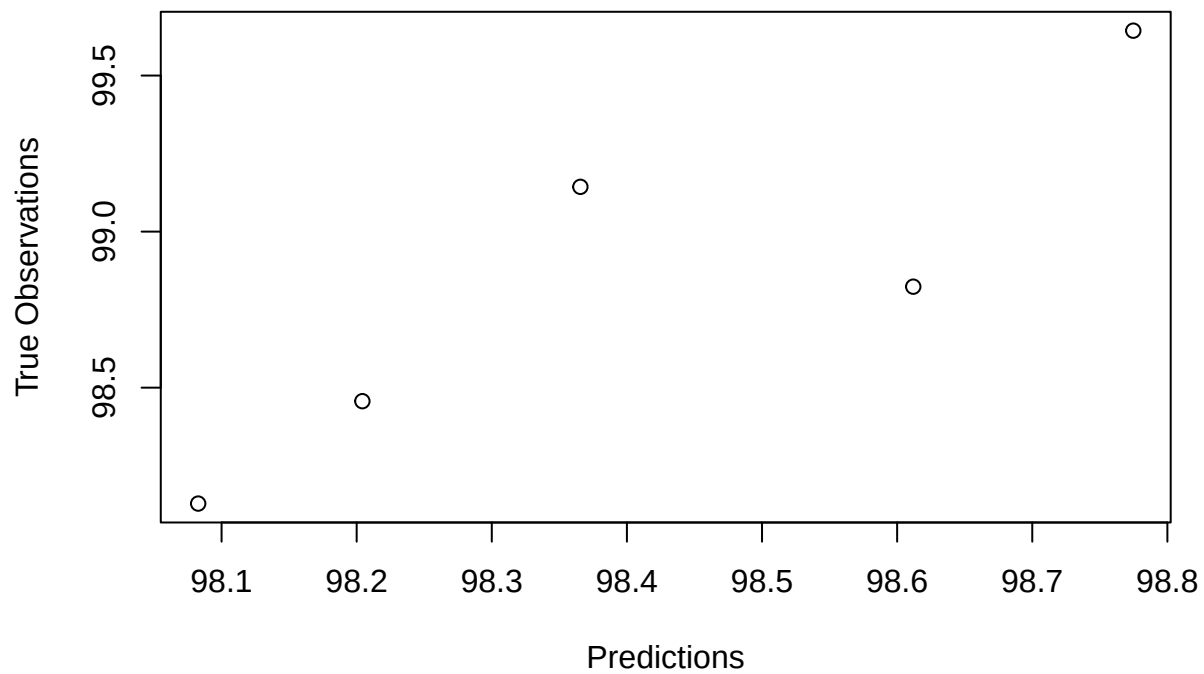
$$\nu_{t+1} = \nu_t + \beta \quad (10)$$

```

# Model 1 CV
data_stan1 <- list(TT = length(y), y = y, z0 = 96)

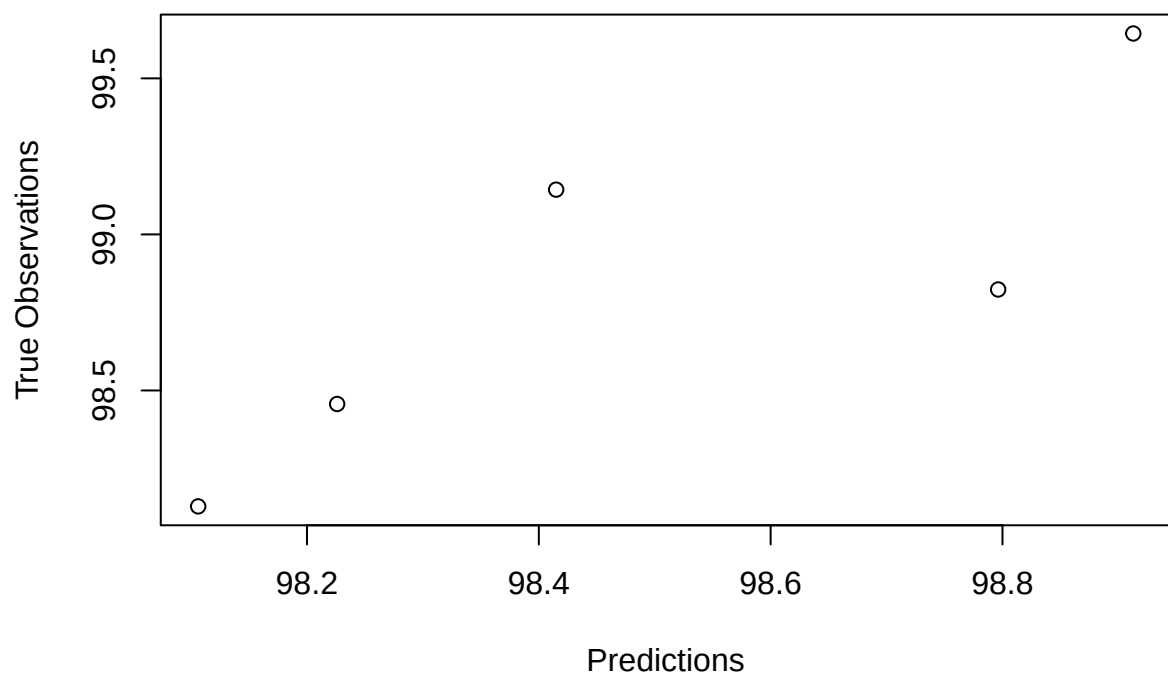
cv1 <- stint_CV("Linear_SSM.stan", data_stan1)

```



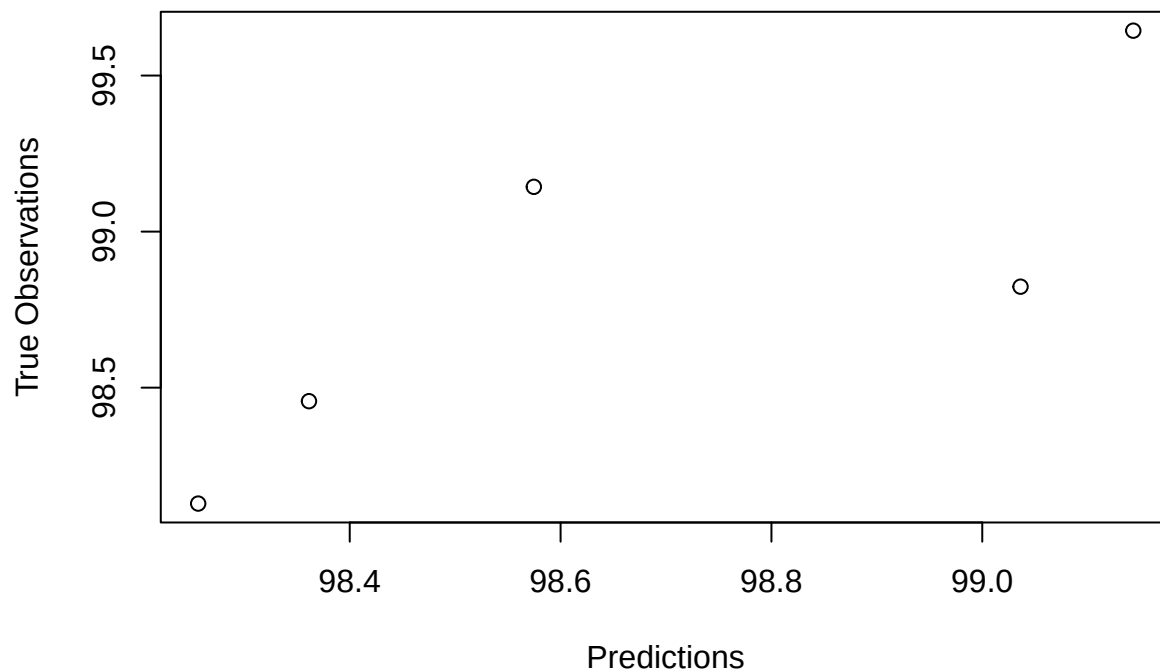
```
# Model 2 CV
data_stan2 <- list(TT = length(y), y = y, z0 = 96)

cv2 <- stint_CV("Basic_SSM2.stan", data_stan2)
```



```
# Model 3 CV
data_stan3 <- list(TT = length(y), y = y, z0 = 96, v0 = .01, sdo0 = .3)

cv3 <- stint_CV("Increasing_Slope_SSM.stan", data_stan3)
```



```
# Model 1 RMSPE  
cv1[[1]]
```

```
## [1] 0.5423718
```

```
# Model 2 RMSPE  
cv2[[1]]
```

```
## [1] 0.4731482
```

```
# Model 3 RMSPE  
cv3[[1]]
```

```
## [1] 0.3590789
```

Conclusion

The full model performs best here most of the time, which makes sense since that is the model that we are simulating from. Overall, I think the important thing here is that it gives a framework to do cross-validation for model selection on real data.